

Story to Krea 2 Prompt Batch

A ComfyUI custom node — installation from a brand-new ComfyUI setup, usage examples, and troubleshooting

This node reads a story from a **.txt file** and uses a **local GGUF language model** (via llama-cpp-python — no cloud API, no key, no network calls) to generate a batch of **Krea 2 image prompts**: one long, descriptive natural-language prompt per key visual moment of the story. Krea 2 responds to coherent prose and context rather than keyword stuffing, so every prompt is written as flowing sentences following *[Subject] + [Action/Pose] + [Scene/Location] + [Lighting] + [Color] + [Composition & Camera] + [Style/Medium]*.

- The LLM reads the whole story and **picks the most visually compelling moments itself**.
- Recurring characters and settings get **one consistent description** reused across the whole batch, with optional **LoRA trigger words** woven naturally into the prose.
- Prompts come out **both ways**: an in-graph STRING list for direct batch generation, and numbered .txt files on disk.
- The language model stays **resident in memory** between runs — it reloads only when you change the model file or its settings.

In this guide: Part 1 – installing ComfyUI from scratch • Part 2 – installing this node • Part 3 – usage examples
• Part 4 – troubleshooting

Part 1 — Installing ComfyUI from scratch (Windows)

Skip this part if ComfyUI already runs on your machine. This guide assumes the Windows **portable** build with an NVIDIA GPU — the most common setup (the node also works on CPU, Linux, and the ComfyUI Desktop app).

What you need

- Windows 10/11, 64-bit.
- An NVIDIA GPU with 8 GB+ VRAM is recommended. CPU-only works — just slower.
- About 20 GB of free disk space: ComfyUI itself, one ~5 GB GGUF language model, and your Krea 2 image model.
- 7-Zip (or any tool that opens .7z archives).

Step 1.1 — Download and extract ComfyUI

Download the portable build from the official releases page and extract it somewhere with plenty of space (for example C:\ComfyUI):

```
https://github.com/comfyanonymous/ComfyUI/releases/latest
file: ComfyUI_windows_portable_nvidia.7z
extract to: C:\ComfyUI\ (creates C:\ComfyUI\ComfyUI_windows_portable\)
```

Step 1.2 — First launch

Inside **ComfyUI_windows_portable**, double-click **run_nvidia_gpu.bat** (or **run_cpu.bat** if you have no NVIDIA GPU). A console window opens, and after startup your browser loads the ComfyUI interface at:

```
http://127.0.0.1:8188
```

Keep the console window open while ComfyUI runs — it is also where this node prints its progress logs.

Step 1.3 — Know the folders

Everything below is relative to **ComfyUI_windows_portable\ComfyUI**:

Folder	What goes there
custom_nodes\	Custom node packages — this node's folder goes here.
models\checkpoints\	Your Krea 2 image model (and other image checkpoints).
models\loras\	Character/style LoRA files referenced in your map.
models\LLM\	GGUF language models used by this node (the folder is created automatically on first launch).
input\	Story .txt files — the node's story dropdown reads from here.
output\story_prompts\	Where the generated prompt .txt files are written.

Part 2 — Installing the Story Prompts node

Step 2.1 — Copy the node into custom_nodes

Copy the whole **Story_Prompts_Node** folder into **ComfyUI\custom_nodes** so you end up with:

```
ComfyUI_windows_portable\ComfyUI\custom_nodes\Story_Prompts_Node\  
  __init__.py  story_prompt_node.py  llm_engine.py  requirements.txt  ...
```

If the node lives in a git repository you can instead run “git clone <repo-url>” inside the custom_nodes folder.

Step 2.2 — Install the dependency into ComfyUI's own Python

The portable build ships its own Python in **python_embeded**. The single most common install mistake is running plain “pip install”, which targets the wrong Python. Open a terminal **in the ComfyUI_windows_portable folder** and run:

```
.\python_embeded\python.exe -m pip install -r .\ComfyUI\custom_nodes\Story_Prompts_Node\requirements.txt
```

This installs llama-cpp-python. The default build runs the LLM on **CPU only** — fully functional, just slower.

Step 2.3 — Optional but recommended: GPU build (NVIDIA)

To run the language model on your GPU, replace the default build with a prebuilt CUDA wheel:

```
.\python_embeded\python.exe -m pip install llama-cpp-python --extra-index-url https://abetlen.github.io/llama-cpp-python
```

Pick the index matching your CUDA runtime (cu121 / cu122 / cu124). If no wheel fits your system, keep the CPU build and set the node's **gpu_layers** to 0.

Step 2.4 — Download a GGUF language model

Use an **instruct-tuned, uncensored (“abliterated” / HERETIC)** GGUF. A standard aligned model can refuse or water down individual scenes partway through an unattended batch; an uncensored variant writes every prompt it is asked for. Recommended — the same model artfat's LLM prompter recommends (fully uncensored thinking model):

```
https://huggingface.co/DavidAU/
  Qwen3.5-9B-Claude-4.6-OS-Auto-Variable-HERETIC-UNCENSORED-THINKING-MAX-NEOCODE-Imatrix-GGUF
file:      the Q4_K_M quant (~6.8 GB); Q4_K_S also works
place in:  ComfyUI\models\LLM\
```

It is a **thinking model**: set **max_tokens_per_prompt** to ~600–800, because its reasoning spends from that budget before the prompt appears (the node strips the think blocks from all outputs automatically). Lighter, non-thinking alternative — faster, and the default token budget is fine:

```
https://huggingface.co/mlabonne/Meta-Llama-3.1-8B-Instruct-abliterated-GGUF
file:      meta-llama-3.1-8b-instruct-abliterated.Q4_K_M.gguf  (~4.9 GB)
place in:  ComfyUI\models\LLM\
```

huihui-ai on Hugging Face maintains abliterated builds of most major model families. Keep whatever you pick instruct-tuned — base models ramble. Unlike artfat's node there is no chat-handler dropdown here (the GGUF's embedded chat template is used automatically) and no mmproj file is needed — this node is text-only.

Step 2.5 — Add a story

Copy the bundled sample into ComfyUI's input folder (or drop in any story of your own as plain UTF-8 .txt):

```
copy custom_nodes\Story_Prompts_Node\examples\sample_story.txt  ComfyUI\input\
```

Step 2.6 — Restart and verify

Restart ComfyUI, then double-click the canvas and search for “story”. You should see the node **Story - Krea 2 Prompt Batch** (category prompt/story). Verify:

- The node appears in search and can be added to the canvas.
- **story_file** lists sample_story.txt and **gguf_model** lists your .gguf file (press **R** in ComfyUI to refresh if you added files while it was running).
- The console shows no import errors mentioning Story_Prompts_Node.

Part 3 — Usage

The node at a glance

Input	What it does
story_file	The story .txt, picked from ComfyUI\input (subfolders included).
num_prompts	How many prompts to generate (1–200). A short story may yield fewer — the node never pads with filler.
gguf_model	The local GGUF language model from ComfyUI\models\LLM.
context_length / gpu_layers	LLM context window, and GPU offload (-1 = all layers on GPU, 0 = CPU only).
temperature, top_p, top_k, repeat_penalty	Sampling for the creative writing pass. Analysis passes automatically run at temperature ≤ 0.5.
max_tokens_per_prompt	Length cap per generated prompt (~2–5 sentences at the default of 200). Thinking models need 600+.
seed	Fix to reproduce a batch exactly; each scene derives its own sub-seed.
style_medium / style_custom	Dropdown of style presets — 10 SFW and 5 NSFW options. “none” = the LLM infers a style per scene; “custom” = type your own into style_custom. See Example 3.
character_lora_map	One line per recurring character/setting — see Example 2. Ships with a commented template showing the format.
system_instructions	The prompt-enhancer rules sent to the LLM: priority rules, NSFW handling, and the Krea 2 prose formula. Pre-filled and editable directly on the node.
output_dir / filename_prefix	Where and how the prompt .txt files are written. Reruns overwrite.
enable	Off = skip the LLM entirely and return the previous batch instantly.

Output	Description
prompts (STRING list)	The generated prompts in story order. Because it is a list , every downstream node runs once per prompt — one queue press generates the whole image batch.
prompt_count (INT)	How many prompts were actually generated.
character_bible (STRING)	The resolved character/setting descriptions used across the batch — useful for inspection.
saved_paths (STRING list)	Disk paths of the written .txt files.

Example 1 — Your first batch

Add the node, set **story_file** = sample_story.txt, pick your **gguf_model**, set **num_prompts** = 5, and press **Queue**. Nothing needs to be connected — the node is an output node and runs on its own. The console shows the three passes:

```
[StoryPromptBatch] pass 1/3: building the character/setting bible
[StoryPromptBatch] pass 2/3: selecting up to 5 key visual moments
[StoryPromptBatch] pass 3/3: writing 5 Krea 2 prompts
[StoryPromptBatch] scene 1/5: A young woman steps onto a swaying rope bridge...
[StoryPromptBatch] wrote 5 prompt files to ...\output\story_prompts
```

Open **output\story_prompts** and you'll find scene_01.txt through scene_05.txt. A generated prompt looks like this (illustrative):

"A young woman with short auburn hair and a weathered brown leather jacket steps onto a swaying rope bridge above a black river gorge, gripping the frayed guide rope with one hand and pressing the other to a brass key hung at her chest. The first light of dawn breaks over the ridge behind her, rimming the mist in gold while the water far below stays cold and dark. Shot from across the gorge in a wide cinematic composition, she is small against the vast drop, in the style of moody atmospheric photography."

Example 2 — Consistent characters with LoRAs

Krea 2 has no memory between generations, so the node keeps a **character/setting bible**: one description per recurring character, reused in every prompt they appear in. Paste entries like these into **character_lora_map** (one per line, fields separated by | — everything after the name is optional):

```
Mira | mira_char_v2.safetensors | mira_xj23 | a young woman with short auburn hair, green eyes, and a weathered brown leather jacket
Jonas | | a wiry old ferryman with a silver beard and a patched gray oilskin coat
Old Mill
```

- **Description given** → used verbatim in every scene that character appears in.
- **Description blank** (like Old Mill) → the LLM writes one from the story and keeps it consistent.
- **Trigger word given** (mira_xj23) → woven naturally into the prose of each of that character's scenes, so the LoRA fires. If the LLM drops it, the node prepends it.
- **LoRA filename** is bookkeeping for you — load the LoRA itself in the workflow with a normal Load LoRA node.

A more detailed entry shows how specific a good description can get — the node's map field ships with entries like these as a commented template (delete the leading # to activate a line):

```
Lena | lena_flux_v3.safetensors | lena_v3 | a curvy young woman in her mid-20s with long
    platinum-blonde hair, blue eyes, a small star tattoo on her left collarbone, and a
    fitted black slip dress
Marcus | | a tall broad-shouldered man with a trimmed dark beard, a buzz cut, and a gray
    henley with rolled-up sleeves
Neon Loft | loft_env_v1.safetensors | neonloft | a downtown loft at night with exposed
    brick, pink-and-blue neon signs, and rain-streaked floor-to-ceiling windows
```

In the actual field each entry is ONE line — the wrapping above is only for print. The more specific the description (age, build, hair, eyes, distinguishing marks, signature clothing), the more consistent the character looks across the whole batch.

Wiring the batch into a Krea 2 workflow

- 1. On a **CLIP Text Encode** node, right-click the *text* widget and choose **Convert widget to input**.
- 2. Connect this node's **prompts** output to that new text input.
- 3. Build the rest of the graph as usual: Krea 2 checkpoint loader → Load LoRA node(s) for every LoRA in your map → sampler → VAE decode → Save Image.
- 4. Press Queue once. Because prompts is a list, the sampler chain runs once per prompt — the whole story renders as a batch.

Prefer file-based batching? Point any “load prompts from directory” loader at `output\story_prompts` instead — the files are always written as well.

Example 3 — Picking a style for the batch

style_medium is a dropdown that locks the Style/Medium slot of the formula for every prompt in the batch. It ships with **10 SFW presets** — cinematic film still, moody atmospheric photography, golden hour portrait photography, 35mm analog film photo, watercolor storybook illustration, classical oil painting, anime cel shading, dark fantasy digital painting, comic book graphic novel art, cinematic 3D render — and **5 NSFW presets** — boudoir photography, sensual glamour photography, artistic nude photography, erotic digital painting, hentai anime style. Leave it on “none” to let the LLM choose a fitting medium per scene, or pick “custom” and type anything into the **style_custom** field.

The NSFW presets assume the uncensored model from Step 2.4 — a standard aligned model may soften those scenes. For a consistent look across the whole batch, picking one style is recommended over “none”.

Everyday tips

- **Reproducibility:** set the seed control to *fixed* to regenerate the exact same batch; *randomize* for variety each run.
- **Iterating downstream:** flip **enable** off while you tweak samplers — the node returns its previous batch instantly without touching the LLM.
- **Editing the story:** the node hashes the story file, so saving changes to the .txt automatically re-triggers generation on the next queue.
- **Long stories:** anything over the context budget is chunk-summarized first (watch for “compression round” lines in the console). Raising **context_length** to 16384+ reduces how much gets compressed.
- **Custom rules:** the Krea 2 guidance lives in the editable **system_instructions** box — tweak it freely; your changes apply on the next run.

Part 4 — Troubleshooting

Almost every problem shows its cause in the **console window** (the terminal opened by `run_nvidia_gpu.bat`). This node logs everything under the tag **[StoryPromptBatch]** — passes, the bible, per-scene previews, warnings, and file paths.

Symptom	Fix
Node doesn't appear in search after restart	The dependency isn't installed into ComfyUI's own Python. Re-run the Step 2.2 command exactly (it must start with <code>.python_embeded\python.exe</code>), then restart. Check the startup console for an import error mentioning <code>Story_Prompts_Node</code> .
Error: "llama-cpp-python is not installed"	Same cause as above — plain "pip install" targeted your system Python instead of <code>python_embeded</code> . Use the Step 2.2 command.
CUDA wheel install fails / no matching wheel	Try the other indexes (<code>cu121</code> , <code>cu122</code> , <code>cu124</code>). If none fit, keep the default CPU build and set <code>gpu_layers = 0</code> — slower, but everything works.
Out of VRAM (LLM load fails, or image sampling crashes afterwards)	The LLM stays resident by design. Lower <code>gpu_layers</code> (<code>0</code> = pure CPU), use a smaller quantization (Q4 instead of Q8) or a smaller model, or generate prompts first and then flip enable off — the cached batch is returned without reloading the LLM while you render images.
story_file or gguf_model dropdown is empty	Stories belong in <code>ComfyUI\input</code> (as <code>.txt</code>), models in <code>ComfyUI\models\LLM</code> (as <code>.gguf</code>). Press R in ComfyUI to refresh after adding files.
Prompts ramble, contain labels like "Subject:", or come out as keyword lists	Use an instruct-tuned GGUF — base models drift. Lowering temperature to <code>~0.7</code> helps. The node scrubs leaked labels automatically, but a better model beats post-processing.
A scene comes back as a refusal or noticeably softened	You're running a standard aligned model. Switch to the uncensored/ablated GGUF from Step 2.4 — unattended batch generation needs a model that writes every scene it's given.
Thinking model returns empty prompts / "scene produced an empty prompt" warnings	The model's reasoning ate the whole token budget before the answer started. Raise <code>max_tokens_per_prompt</code> to <code>600+</code> . (The node strips think blocks from the output automatically.)
Error: "unknown model architecture" when the model loads	Your <code>llama-cpp-python</code> predates the model family (e.g. Qwen 3.5). Upgrade it with <code>.python_embeded\python.exe -m pip install -U llama-cpp-python</code>
Fewer prompts than num_prompts	By design: the story didn't contain that many distinct visual moments, and the node refuses to invent filler scenes. The console logs the actual count.
Error: "context_length is too small"	Raise <code>context_length</code> (e.g. <code>16384</code>) or lower <code>num_prompts</code> . Very high <code>num_prompts</code> values need room for the scene list in the context window.
Prompts cut off mid-sentence	Raise <code>max_tokens_per_prompt</code> (e.g. <code>300–400</code>).
Generation is very slow	You're on the CPU build or <code>gpu_layers = 0</code> . Install the CUDA wheel (Step 2.3) and set <code>gpu_layers = -1</code> . A smaller/more-quantized model also speeds things up.
Same prompts every run	The seed control is set to fixed. Set <code>control_after_generate</code> to randomize — or keep fixed and edit the story/settings; the node re-runs automatically on change.
A LoRA trigger word is missing or doubled	If the LLM drops a trigger, the node prepends it and logs a note — the LoRA still fires. If you see it twice, remove the trigger word from your description field (it belongs only in the trigger column of the map).

Symptom	Fix
“scene selection failed twice — falling back to even story chunks” in the console	The model couldn't follow the scene-list format, so the node split the story evenly instead — results are still usable. A stronger instruct model fixes it properly.

Story to Krea 2 Prompt Batch v1.0.0 • MIT license • architecture inspired by artfat-comfyui-llm-prompter • generated July 2026